

SAP Report

Design and Implementation of a Microservices-Based Application

Rosario Zaccone

June 2026

Version: v0.1.19

Last Modified: 2026-07-09

GitHub Repository: <https://github.com/rosario-zaccone/rione>

Contents

1	Application Description	3
2	Technological Choices	3
2.1	Technologies Used	3
3	Testing Architecture	4
3.1	Types of Tests	5
3.2	CI/CD Pipeline	5
4	Domain-Driven Design Approach	6
4.1	Strategic Design	6
4.1.1	Bounded Contexts	6
4.1.2	Ubiquitous Language	6
4.2	Tactical Design	7
5	Requirements	7
5.1	User Stories	8
5.2	Requirements Tables	8
5.3	Business Rules	8
6	Implementation and Deployment Architecture	9
6.1	Clean and Hexagonal Service Structure	9
6.2	Event-Driven Communication	9
6.3	Application Security	11
7	Main Microservices	11
7.1	API Gateway	11
7.2	User Service	12
7.3	Social Service	14
7.4	Post Service	16
7.5	Notification Service	18

8	Additional Patterns and Monitoring	19
8.1	Health API	19
8.2	Circuit Breaker	19
8.3	Application Metrics	20
9	Frontend	21
10	Implementation	23
10.1	Use of AI During Coding	23
11	Conclusions	23

1 Application Description

Rione is a social network centred on small local communities. Its purpose is to support interaction among people who live in the same neighbourhood, limiting the social space to nearby residents rather than creating a broad and generic online network. The platform is based on the idea that digital communication can become more useful and meaningful when it is connected to the places in which people live their everyday lives.

The core principle of the project is neighbourhood-based interaction. Users can see and interact only with other users who belong to the same neighbourhood. For example, in a city such as Bologna, neighbourhoods may include Navile, Saragozza, San Vitale, Savena, and others. A user who belongs to Navile should interact only with other users from Navile, while users from different neighbourhoods remain outside that local social space. This constraint gives the platform a clear community boundary and distinguishes it from traditional social networks, where interactions are often global, anonymous, or weakly connected to real-life contexts.

The motivation behind Rione is to create a more local and purposeful form of on-line participation. By focusing on neighbourhood communities, the platform aims to strengthen relationships among nearby residents, encourage participation in local community life, and make online exchanges more relevant to practical daily needs. A local social network can help people share information, ask for help, discuss neighbourhood issues, organize small events, and become more aware of what happens around them.

Another important motivation is trust. Interactions among people who share the same local environment can be more concrete than interactions in large-scale social networks. The neighbourhood boundary encourages users to perceive other participants not as distant strangers, but as members of the same local community. This can increase the perceived value of contributions and support more responsible communication.

The platform also introduces deliberate constraints to promote more thoughtful participation. For instance, posts and comments may require a minimum length, encouraging users to write meaningful contributions instead of very short or superficial messages. These constraints are intended to increase attention, reduce low-effort interactions, and improve the quality of discussions. In this sense, Rione is designed not only as a communication tool, but also as a way to encourage more careful, relevant, and community-oriented digital participation.

2 Technological Choices

This section describes the main technologies and development tools used in the project. Architectural and design decisions are discussed in later sections.

2.1 Technologies Used

The project is implemented as a Maven multi-module application. The backend uses Java and Spring Boot, while local execution and infrastructure are provided through Docker Compose.

- **Java 21**: main programming language for backend services.
- **Spring Boot 4.1.0**: base framework for the backend microservices.

- **Spring Web MVC**: implementation of REST APIs exposed by the services.
- **Spring Data JPA**: persistence layer for relational data.
- **PostgreSQL**: relational database used by the services that require persistence.
- **Apache Kafka**: event broker used for asynchronous communication between social, post, and notification workflows.
- **Spring Security**: authentication and authorization through bearer tokens and service-to-service security rules.
- **Springdoc OpenAPI**: generation and publication of API documentation.
- **React and Vite**: frontend implementation and local development environment.
- **Docker and Docker Compose**: containerization and local orchestration of services, databases, Kafka, Prometheus, Grafana, and pgAdmin.
- **Prometheus and Grafana**: collection and visualization of technical and business metrics.
- **Maven**: build, dependency management, and test execution.
- **GitHub and GitHub Actions**: version control and continuous integration.

3 Testing Architecture

The testing strategy is organized around Test-Driven Development and layered verification. Business rules are first expressed in requirements documents and then mapped to executable tests. The goal is not only to check implementation details, but also to keep the code aligned with the expected domain behavior.

The project follows the idea of the testing pyramid. At the base of the pyramid there are many fast unit tests, because they are cheap to execute and give quick feedback on domain and application logic. In the middle there are integration tests, which are fewer because they involve frameworks, adapters, persistence, security, messaging, or HTTP boundaries. At the top there are higher-level acceptance tests, which are more expensive but validate business behavior from the user's point of view. Architecture tests are used as an additional technical guardrail to verify that the intended code structure is preserved.

Each main business service follows the same testing structure under `src/test`. The project separates acceptance tests, architecture tests, integration tests, and unit tests. This separation keeps each test focused on a specific kind of risk: business correctness, architectural conformance, adapter behavior, or isolated domain and application logic.

The test suite is executed with Maven. During continuous integration, GitHub Actions runs `verify` on each service and then on the full Maven reactor. This means that a change is validated both in its own module and in the context of the whole project.

3.1 Types of Tests

The project uses four main test layers.

- **Unit Tests:** unit tests verify small units of behavior in isolation. They are placed at the base of the testing pyramid because they are fast, deterministic, and numerous. In this project, they are implemented with JUnit and Mockito and focus on domain models, value objects, and application services. Examples include tests for user value objects such as biography, birth date, and password, and service tests for user, social, post, and notification application logic.
- **Integration Tests:** integration tests verify that multiple components work correctly together across technical boundaries. They occupy the middle level of the testing pyramid because they are slower than unit tests but provide stronger confidence in framework and adapter behavior. In this project, they are implemented with JUnit and Spring testing support and verify REST controllers, security configuration, JWT handling, JPA repositories, Kafka adapters, metrics configuration, and schema migration behavior.
- **Architecture Tests:** architecture tests verify structural constraints rather than business behavior. They check that the implementation respects the intended separation between domain, application ports, application services, and infrastructure adapters. In this project, they are implemented with ArchUnit and act as automated guardrails against accidental dependency violations.
- **Acceptance Tests:** acceptance tests verify the system behavior expected by the requirements. They are placed at the top of the testing pyramid because they validate broader business scenarios. In this project, they are implemented with Gherkin, Cucumber, and JUnit, and they are explicitly modeled on the documented business rules. Feature files use stable rule identifiers such as `USR-BR-001`, `SOC-BR-001`, `PST-BR-006`, and `NOT-BR-001`, so each acceptance scenario can be traced back to a specific business rule.

This combination gives the project broad coverage without relying on a single kind of test. Unit tests keep domain logic fast to verify, integration tests validate the collaboration with frameworks and adapters, architecture tests prevent accidental dependency violations, and acceptance tests protect the business requirements defined in the requirements documentation.

3.2 CI/CD Pipeline

The repository uses GitHub Actions to automatically verify the project. The current pipeline is focused on continuous integration: it checks that the services compile and that their automated tests pass before changes are considered safe to merge.

- The workflow runs on pushes to `feature/**` branches and to `main`.
- The workflow also runs on pull requests targeting `main`.
- A matrix job verifies the main backend services independently: User Service, Post Service, Social Service, and Notification Service.

- Each service job uses Java 21 and executes `./mvnw -B -pl <module> verify`.
- After the service-level jobs, a full Maven reactor verification is executed with `./mvnw -B verify`.
- Maven dependency caching is enabled through the Java setup action to reduce build time.

This pipeline supports a test-first and regression-aware workflow. The goal is to detect broken business rules, architectural violations, integration problems, or compilation errors before they reach the main branch.

4 Domain-Driven Design Approach

The project uses Domain-Driven Design as required by the exam assignment. DDD was adopted to keep the software model close to the business domain of a neighbourhood-based social network, where concepts such as users, neighbourhoods, posts, social relationships, and notifications have precise meanings. This approach also helped separate business decisions from technical details such as REST controllers, persistence, messaging, and infrastructure configuration.

4.1 Strategic Design

Strategic design was used to understand the problem space before defining the implementation structure. The domain was divided into clear business areas, each with its own model, responsibilities, and language. This was useful for identifying service boundaries and for avoiding a single shared model that would mix unrelated concerns.

4.1.1 Bounded Contexts

The analysis started from the overall domain and from conceptual sketches of the main domain components. From this view, four bounded contexts were identified: User, Social, Post, and Notification. Each bounded context owns a specific part of the domain: the User context manages user identity and profile information, the Social context manages neighbourhood membership and relationships, the Post context manages publications and interactions with content, and the Notification context manages the delivery of relevant events to users.

These contexts also correspond to the main business microservices of the system. This mapping keeps each service focused on a coherent set of rules and reduces coupling between services.

4.1.2 Ubiquitous Language

Each bounded context has its own ubiquitous language, documented in the glossary files. The glossary defines the main terms used in requirements, code, tests, and diagrams, so that the same concept is described consistently across the project. This is especially important because similar words can have different meanings in different contexts: for example, a user profile belongs to the User context, while neighbourhood membership belongs to the Social context.

- docs/requirements/user/ubi_language.md
- docs/requirements/social/ubi_language.md
- docs/requirements/post/ubi_language.md
- docs/requirements/notification/ubi_language.md

4.2 Tactical Design

For each bounded context, a UML domain model was designed and is presented later in the report. These diagrams identify the main aggregates, entities, value objects, and relationships used by the services. They provide a bridge between the requirements documentation and the implementation, making the domain structure easier to inspect before looking at the source code.

In the implementation, tactical DDD patterns are used to make the model explicit. Entities represent domain concepts with identity, value objects represent immutable concepts defined by their attributes, and aggregate roots protect consistency boundaries. The code also uses project-specific DDD annotations, such as `DDDEntity`, `DDDValueObject`, and `DDDAggregateRoot`, to make these roles visible and easier to verify.

Value objects are also used to encapsulate validation rules for meaningful attributes. Instead of passing raw strings, numbers, or identifiers through the system, concepts such as e-mail addresses, biographies, names, identifiers, post content, and places are created through dedicated types that validate their own state. This choice keeps invalid values from entering the domain model, avoids duplicating the same checks in controllers or services, and makes the code more expressive because the type itself communicates the meaning and constraints of the value.

Aggregate roots are responsible for the lifecycle of the entities connected to them. External code interacts with the aggregate through the root, while the root creates, updates, removes, or protects internal entities according to the aggregate invariants. For example, comments and reactions are managed through the Post aggregate rather than being modified independently. This keeps related state changes consistent and prevents application services or adapters from bypassing domain rules.

The same idea is reflected in the application structure. Domain rules are kept in the domain model, application services coordinate use cases, and infrastructure adapters handle external concerns such as HTTP, persistence, and messaging. This keeps the core model independent from technical mechanisms and supports the hexagonal organization used by the services.

5 Requirements

The requirements documentation is stored in the following folders:

- docs/requirements/api-gateway/
- docs/requirements/user/
- docs/requirements/social/
- docs/requirements/post/
- docs/requirements/notification/

5.1 User Stories

User stories are short requirement descriptions written from the perspective of an actor. They describe who needs a capability, what they want to do, and why the capability has value.

The user stories are stored in:

- docs/requirements/api-gateway/user_stories.md
- docs/requirements/user/user_stories.md
- docs/requirements/social/user_stories.md
- docs/requirements/post/user_stories.md
- docs/requirements/notification/user_stories.md

5.2 Requirements Tables

Requirement tables translate user stories into structured system responsibilities. They assign stable identifiers to functional requirements, business rules, and non-functional requirements.

The requirement tables are stored in the Software Requirements Specification files:

- docs/requirements/api-gateway/srs.md
- docs/requirements/user/srs.md
- docs/requirements/social/srs.md
- docs/requirements/post/srs.md
- docs/requirements/notification/srs.md

5.3 Business Rules

Business rules are domain constraints that must remain true regardless of the technical interface used to execute an operation. They define when an operation is valid, what must be rejected, and which side effects are allowed.

The business rules are documented in the same requirements folders as the user stories and SRS files:

- docs/requirements/api-gateway/
- docs/requirements/user/
- docs/requirements/social/
- docs/requirements/post/
- docs/requirements/notification/

6 Implementation and Deployment Architecture

The system is implemented as a microservices architecture. Each main bounded context identified during the DDD analysis corresponds to a business microservice: User Service, Social Service, Post Service, and Notification Service. This decomposition keeps each service responsible for a coherent part of the domain and allows each model to evolve without forcing all services to share the same internal representation.

Additional services and infrastructure components were added to support cross-cutting architectural patterns. The API Gateway exposes a single entry point for the frontend and routes requests to the backend services. PostgreSQL databases provide persistence for the services that own durable state, while Kafka is used as the event broker for asynchronous communication. Monitoring and operational components, such as Prometheus and Grafana, are deployed together with the application to observe service health and runtime metrics.

Synchronous communication between microservices is implemented through REST APIs. This is used when a service needs immediate information from another service to complete a use case. To keep this dependency explicit and isolated, services use proxy clients in the infrastructure layer. For example, the Social Service may need user information that goes beyond the user identifier; through a User Service proxy, it can retrieve the required user details without directly accessing the User Service database or sharing its internal domain model. The resulting deployment view is shown in Figure 1.

6.1 Clean and Hexagonal Service Structure

The business microservices follow a clean and hexagonal organization. Their code is divided into domain, application, and infrastructure layers. The domain layer contains the core business model and rules; the application layer coordinates use cases and defines ports; the infrastructure layer contains adapters for REST APIs, persistence, messaging, configuration, and external integrations.

This structure keeps dependencies directed toward the inner layers. Controllers, repositories, Kafka consumers, REST proxy clients, and other technical details depend on the application and domain model, while the domain model does not depend on Spring, databases, HTTP, or Kafka. This makes the business logic easier to test and reduces the impact of changing an external technology. Figure 2 summarizes this dependency direction.

6.2 Event-Driven Communication

An event-driven approach is used where asynchronous communication is more appropriate than direct synchronous calls. In particular, the Notification Service reacts to events produced by other services instead of being called directly by each workflow that may generate a notification.

Kafka acts as the event broker. The Post Service and Social Service publish events on Kafka topics when relevant domain actions occur, and the Notification Service consumes those events to create or deliver notifications. This design reduces coupling between the services that produce business events and the service that handles notifications. It also makes the notification workflow more scalable, because event producers do not need to wait for notification processing to complete before finishing their own use cases.

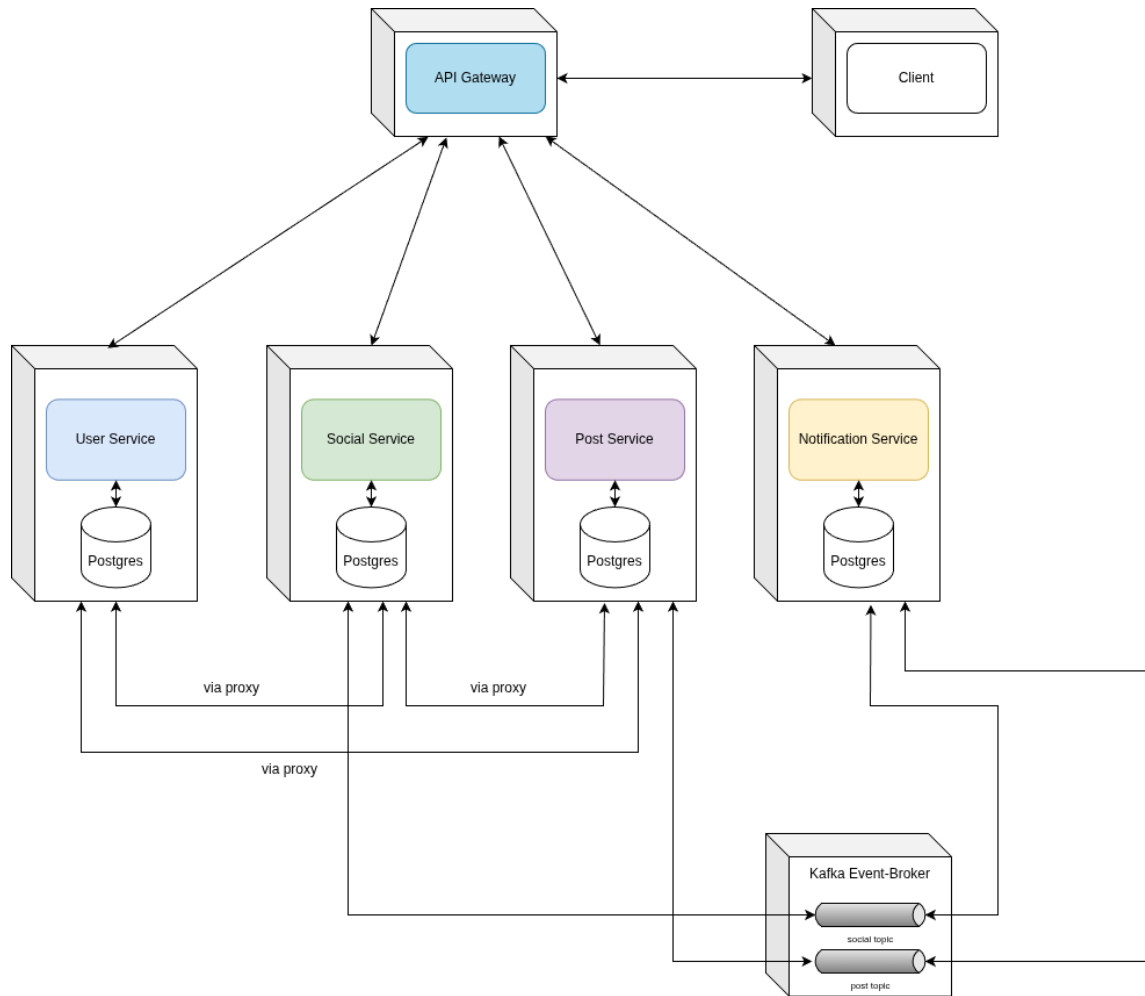


Figure 1: Deployment architecture of the Rione system.

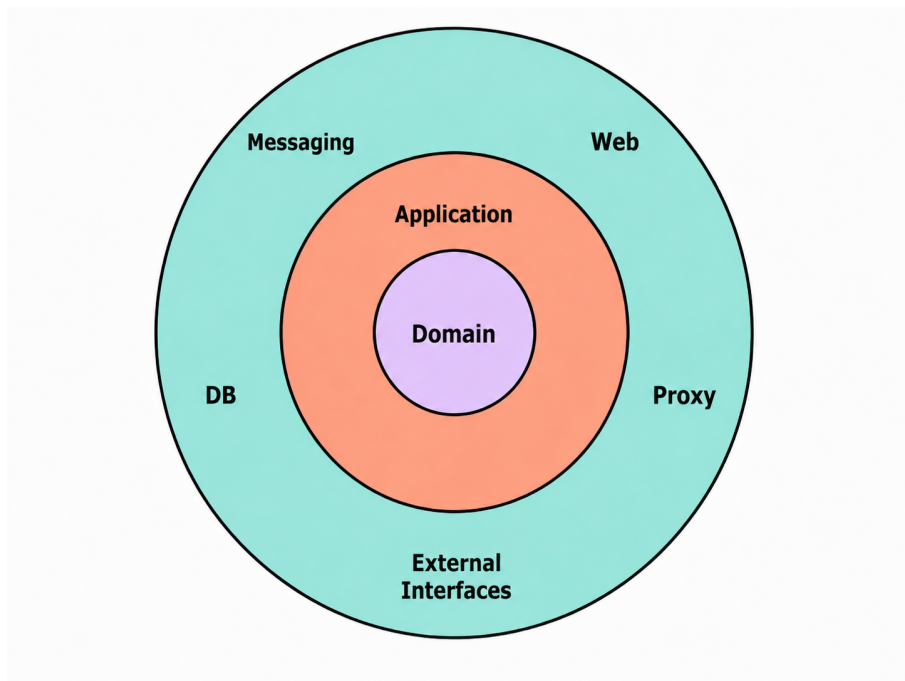


Figure 2: Clean Architecture dependency direction adapted to the Rione service structure.

6.3 Application Security

Security is managed with Spring Security and JWT-based stateless authentication. The User Service is responsible for user registration and login: credentials are validated there, passwords are stored as hashes rather than plain text, and a JWT is returned after a successful login. Clients then send the token in the **Authorization: Bearer <token>** header when calling protected APIs. Each backend service validates the token before executing protected operations, so authentication does not depend on server-side HTTP sessions and can work naturally in a distributed microservices deployment.

The security configuration explicitly separates public, authenticated, administrative, and internal endpoints. Public routes include login, registration, selected read-only resources, health checks, Prometheus scraping, and OpenAPI documentation. Business operations require an authenticated user, and administrative operations such as city management are protected with an admin role. Controllers do not parse tokens directly; instead, JWT validation is handled by a security filter, and controllers retrieve the current user from the Spring Security context. This keeps authentication concerns outside the domain and application logic.

Service-to-service communication is protected with dedicated service tokens. Internal APIs are exposed under `/internal/**` and require `ROLE_SERVICE`; they are intended for backend services, not for frontend clients. For example, the Social Service and Post Service can call the User Service through proxy adapters to retrieve only the minimal user information needed by their use cases. These calls use short-lived service JWTs signed with a separate service secret and validated with service-specific claims such as service identity and target audience. This prevents services from reusing user tokens or accessing another service database directly, while still allowing the system to enforce cross-service rules such as neighbourhood visibility and post privacy.

7 Main Microservices

This section presents the main microservices of the system. Each business service is described through its API contract, domain model, and component-and-connector view. The API diagrams are derived from the OpenAPI specifications in `docs/openapi/`, while the domain diagrams are generated from the PlantUML models in `docs/design/`.

7.1 API Gateway

The API Gateway is the public entry point of the backend. It exposes the routes consumed by the frontend and forwards requests to the internal services, keeping clients independent from the internal service topology. It does not own a domain model; its responsibility is API composition, routing, and boundary-level access to the backend capabilities. Its public API surface is illustrated in Figure 3.

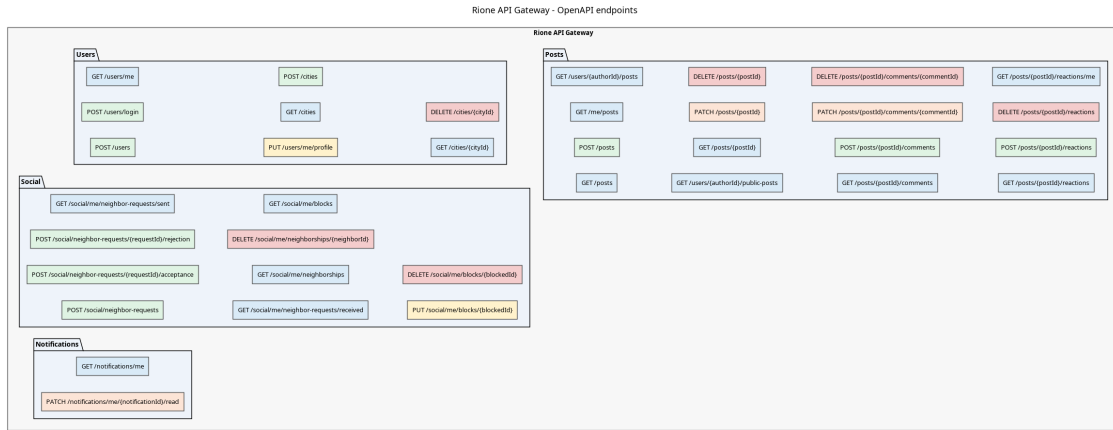


Figure 3: API Gateway OpenAPI diagram.

7.2 User Service

The User Service manages identity and profile-related information. Its domain model includes users, cities, neighbourhoods, and value objects such as user identifiers, names, mail addresses, and biographies. The service also exposes internal endpoints used by other services when they need user-related information without accessing the User Service database directly. Its API contract is shown in Figure 4, its domain model in Figure 5, and its component-and-connector view in Figure 6.

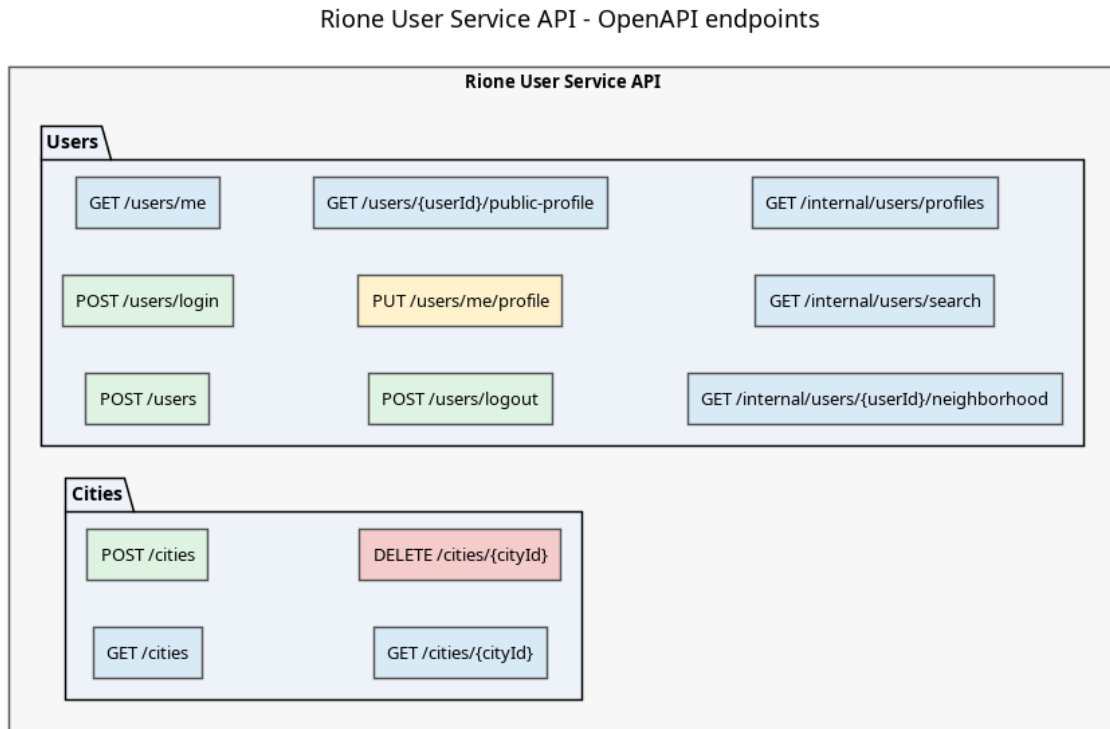


Figure 4: User Service OpenAPI diagram.

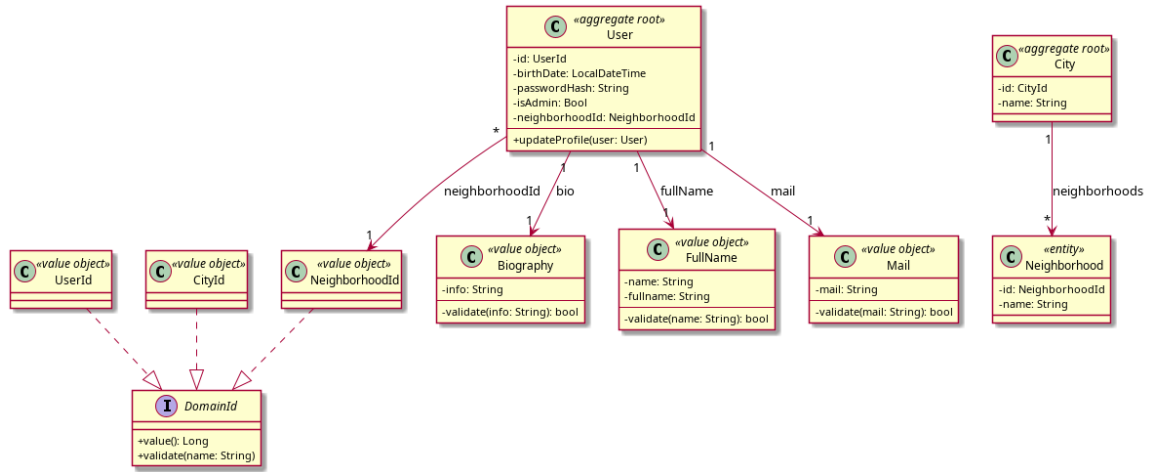


Figure 5: User Service domain model.

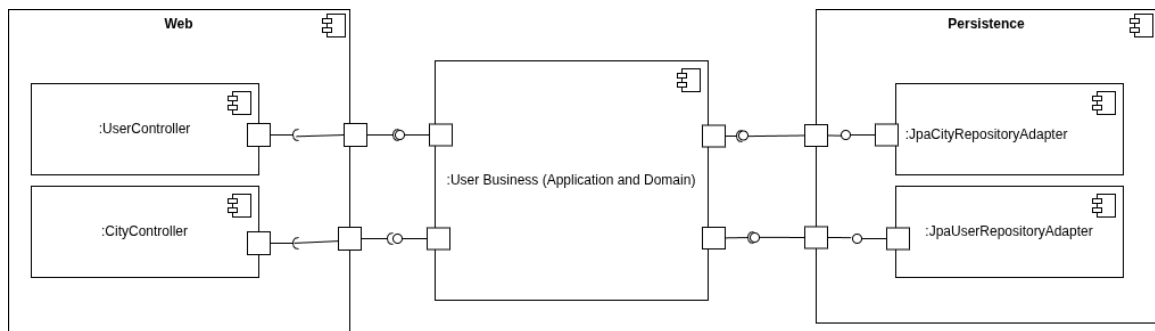


Figure 6: User Service component-and-connector diagram.

7.3 Social Service

The Social Service manages relationships between users inside the local social network. Its domain model focuses on neighbour requests, neighborships, and blocks. It references users by identifier and uses proxy clients when it needs additional user data from the User Service. Its API contract is shown in Figure 7, its domain model in Figure 8, and its component-and-connector view in Figure 9.

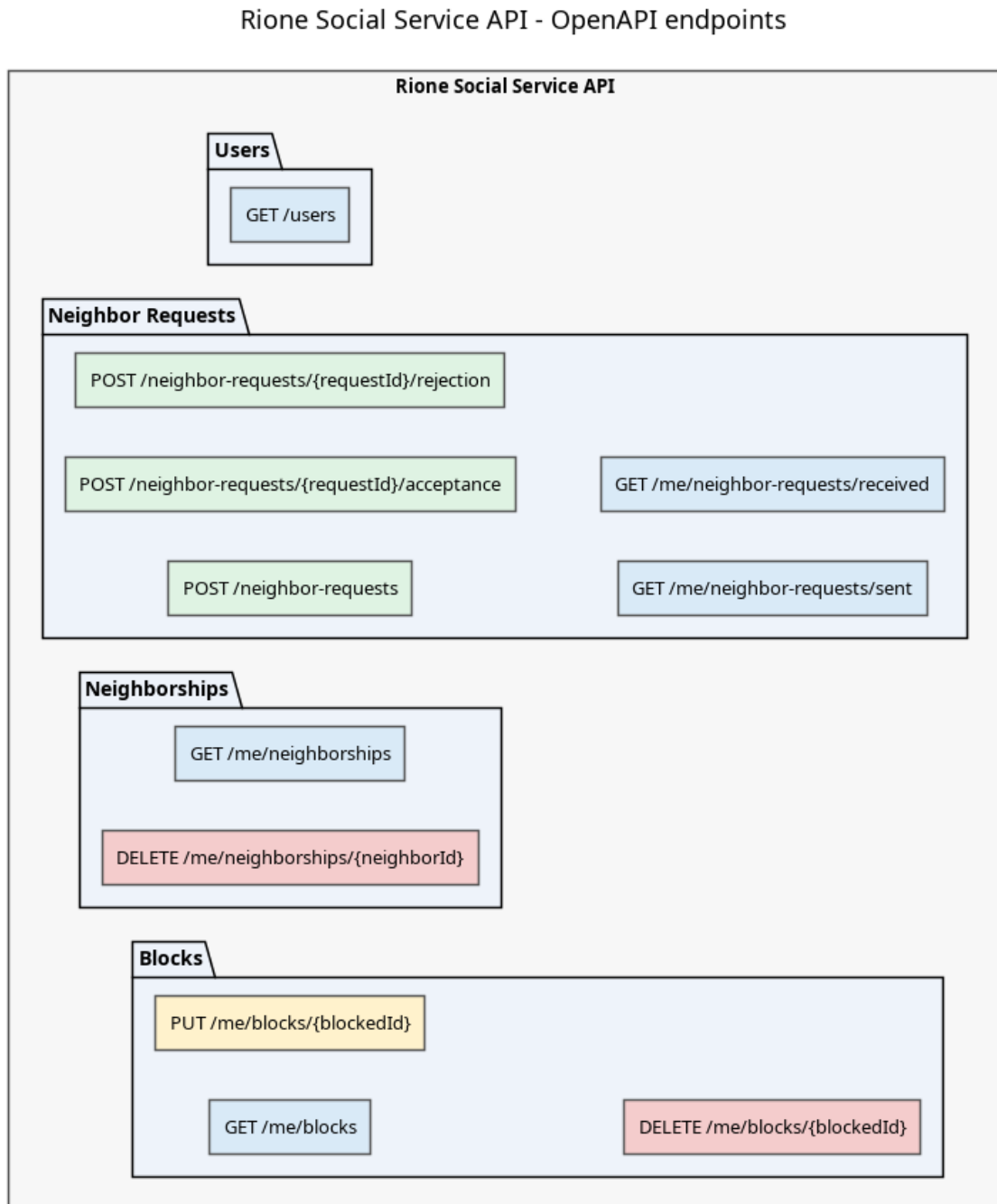


Figure 7: Social Service OpenAPI diagram.

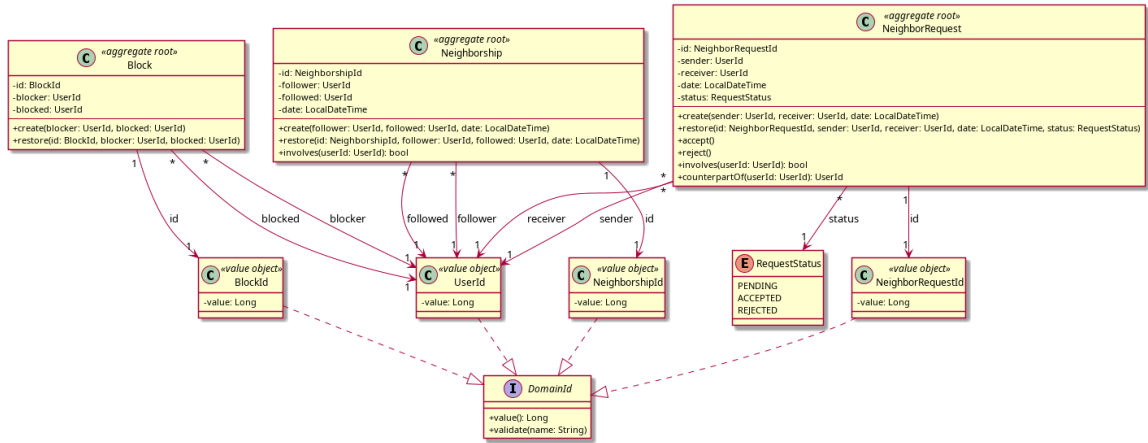


Figure 8: Social Service domain model.

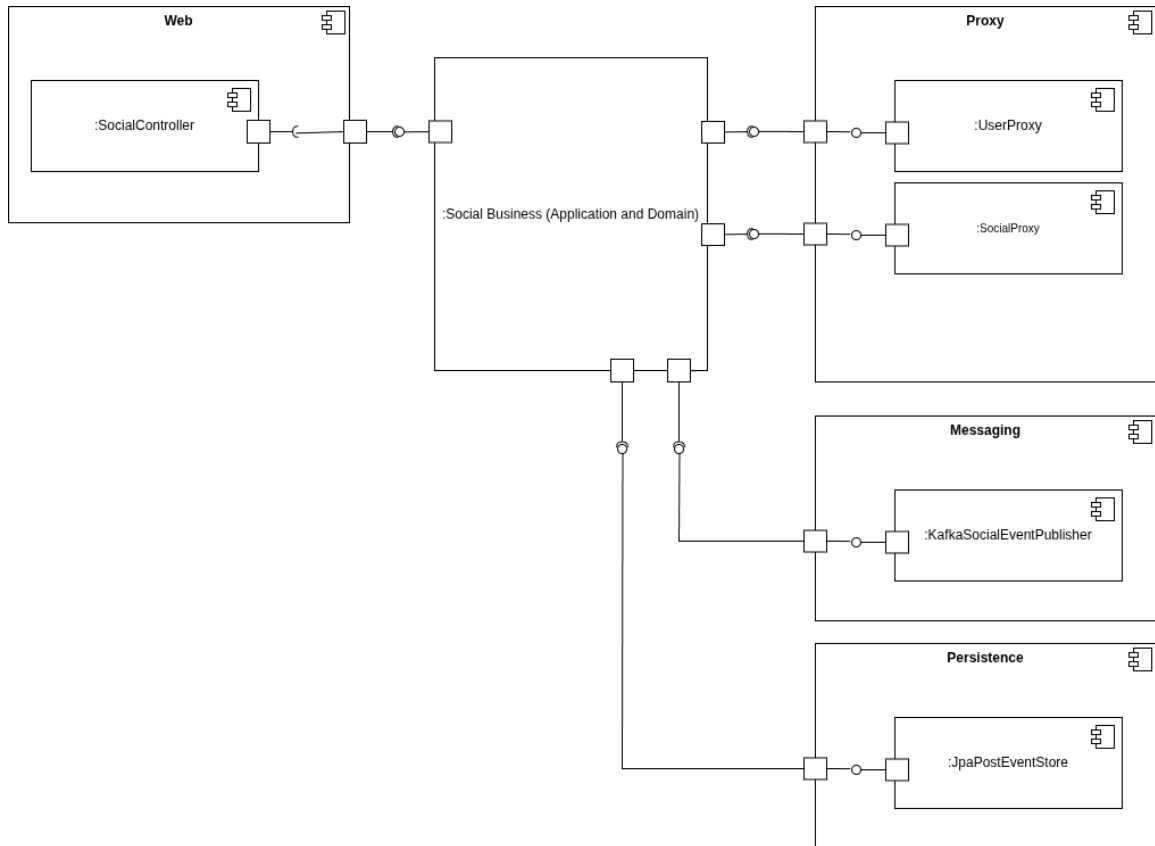


Figure 9: Social Service component-and-connector diagram.

7.4 Post Service

The Post Service manages posts, comments, and reactions. The Post aggregate owns the consistency boundary for content interactions, while comments and reactions are modeled as entities inside the aggregate. The service also publishes events when relevant interactions occur, allowing the Notification Service to react asynchronously through Kafka. Its API contract is shown in Figure 10, its domain model in Figure 11, and its component-and-connector view in Figure 12.

This service implements event sourcing for post state. Instead of storing only the latest snapshot of a post, the service stores an ordered stream of immutable events such as `PostCreated`, `PostContentUpdated`, `PostDeleted`, `CommentAdded`, `CommentUpdated`, `CommentRemoved`, `ReactionAdded`, `ReactionUpdated`, and `ReactionRemoved`. The current state of a post is rebuilt by reading the stream and applying each event in order. This makes the history of changes explicit and allows the service to derive the current post, comments, reactions, score, timestamps, and deleted state from the event sequence.

The event-sourcing boundary is exposed through the `PostEventStore` output port. The application service loads events with `read(postId)` or `readAll()`, reconstructs an in-memory `PostState`, validates the requested operation against that state, and appends one or more new events. The JPA adapter persists events in the `post_events` table with a `postId`, an increasing `version`, an event `type`, an `occurredAt` timestamp, and a serialized payload. Appends use an expected version check; if the stored stream version differs from the expected one, the operation fails with a concurrency error. This gives the Post aggregate optimistic concurrency protection without exposing persistence details to the application core.

Event-sourcing events are kept separate from integration events. The internal post event stream is used to rebuild local state and compute metrics, while notification messages are published separately through the messaging adapter when comments or reactions should notify another user. Replaying the post event stream therefore reconstructs state without triggering external side effects.

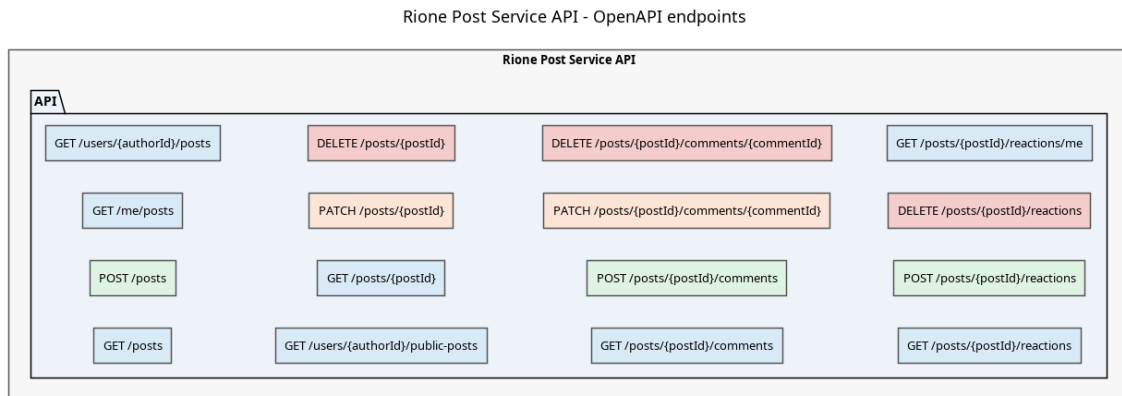


Figure 10: Post Service OpenAPI diagram.

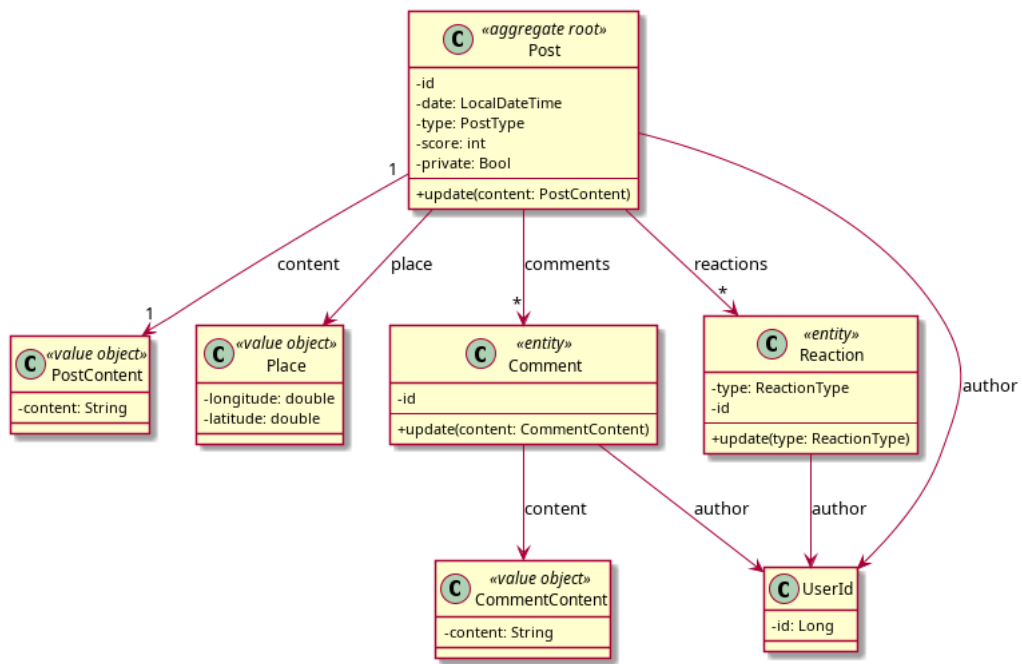


Figure 11: Post Service domain model.

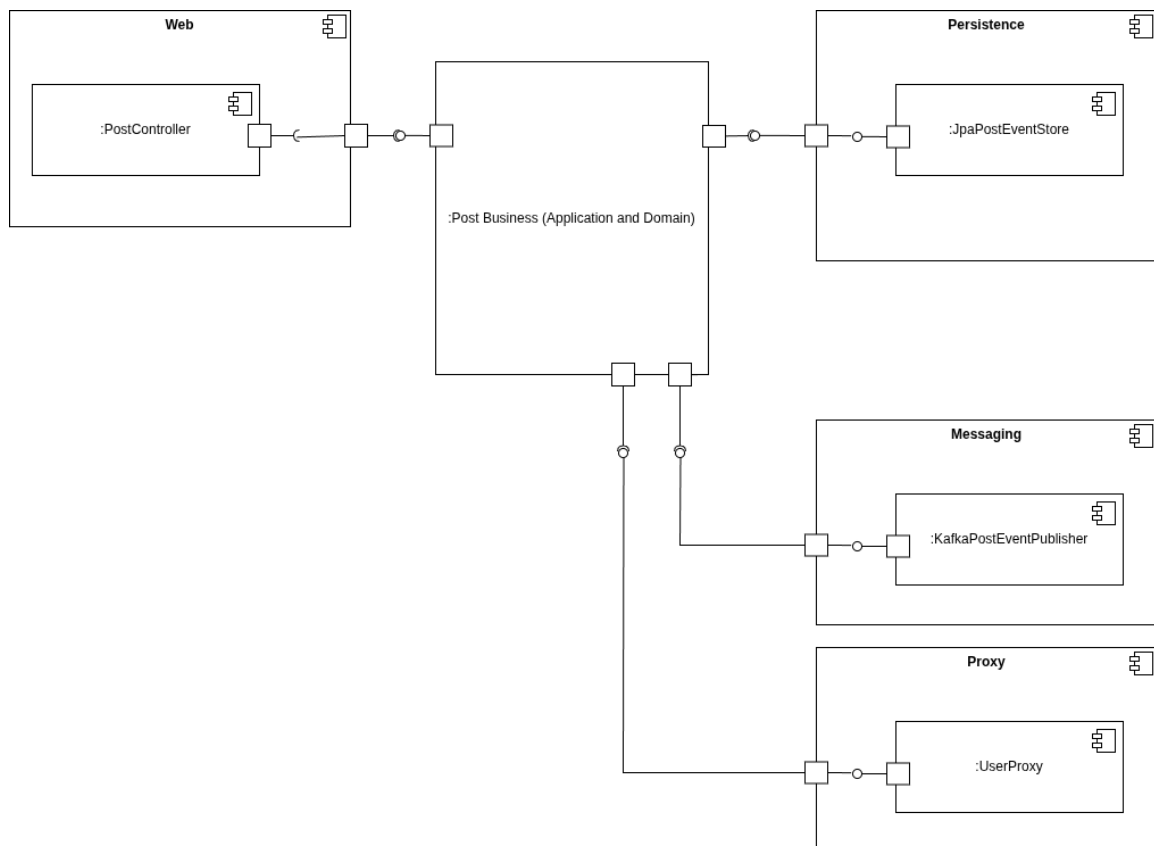


Figure 12: Post Service component-and-connector diagram.

7.5 Notification Service

The Notification Service manages user notifications. Its domain model centers on the Notification aggregate, which records the recipient, actor, notification type, message, occurrence time, and read state. The service consumes Kafka events produced by other services and exposes read-side REST endpoints for retrieving and marking notifications as read. Its API contract is shown in Figure 13, its domain model in Figure 14, and its component-and-connector view in Figure 15.

Rione Notification Service API - OpenAPI endpoints

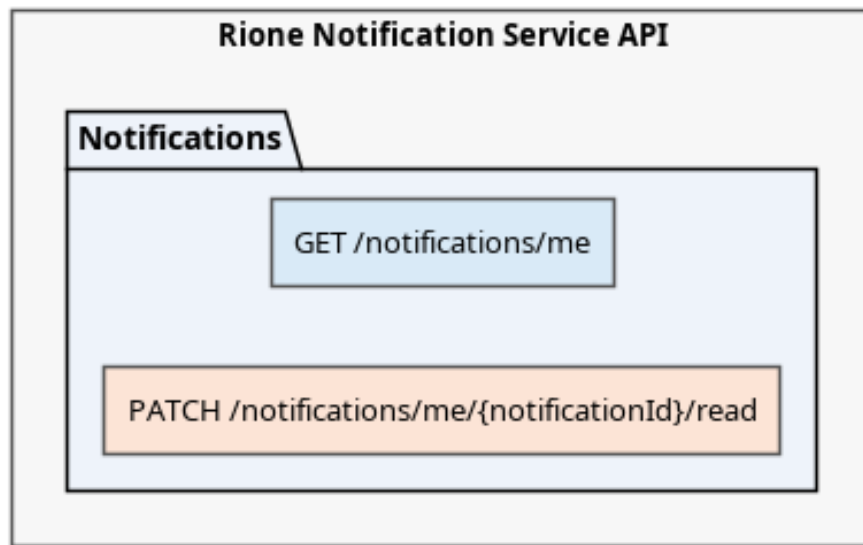


Figure 13: Notification Service OpenAPI diagram.

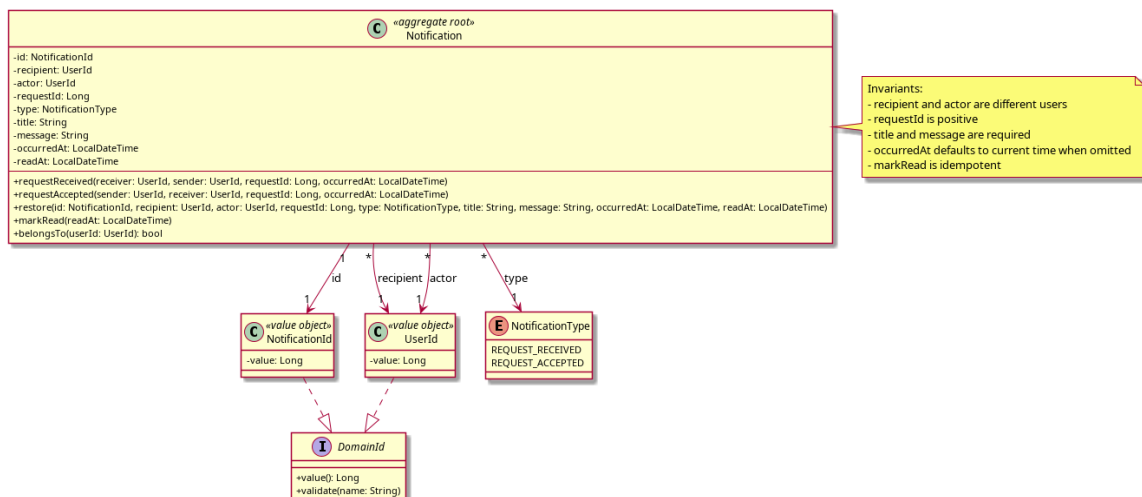


Figure 14: Notification Service domain model.

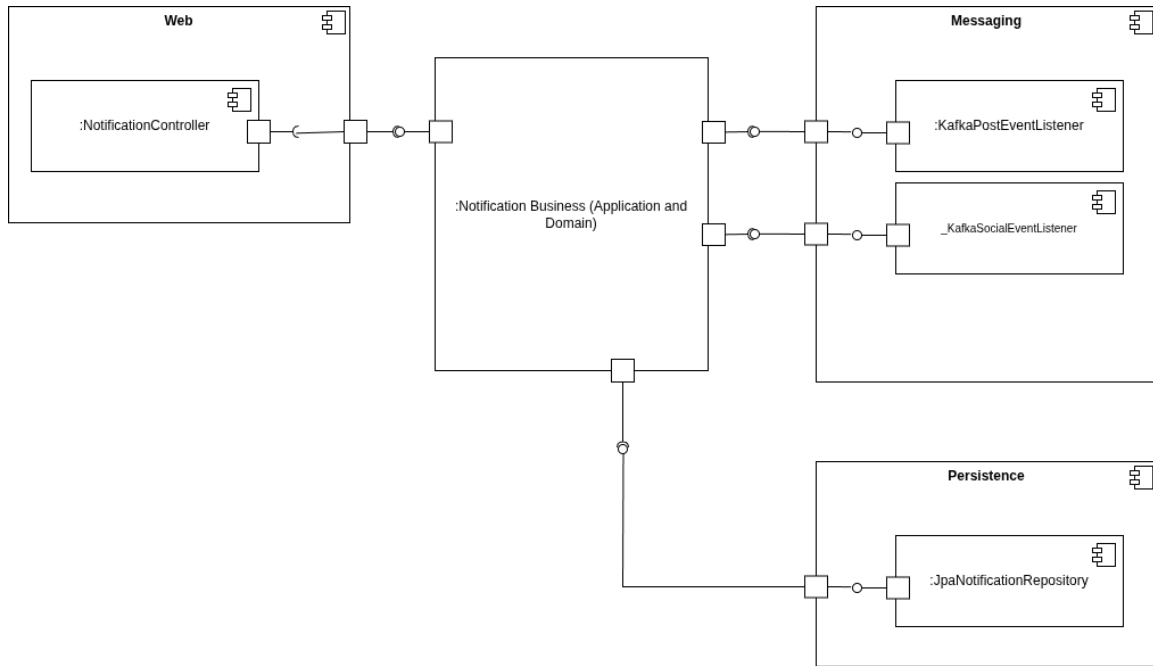


Figure 15: Notification Service component-and-connector diagram.

8 Additional Patterns and Monitoring

In addition to the core microservice, DDD, and hexagonal architecture choices, the project includes operational patterns that make the system easier to run and observe. The most relevant ones are health checks and application metrics. They do not implement business behavior directly, but they are important because a distributed system must expose enough runtime information to understand whether services are alive, reachable, and behaving as expected.

8.1 Health API

Each backend service exposes health information through Spring Actuator. The health endpoint is made available without user authentication so that Docker Compose, monitoring tools, or deployment infrastructure can check whether a service is running and ready to receive traffic. This is especially useful in a microservices architecture, where a failure may affect only one service while the rest of the system continues to run.

The health API is intentionally separated from the business OpenAPI contracts. Business APIs describe domain capabilities such as users, posts, social relationships, and notifications, while Actuator endpoints describe operational state. In the services, health and Prometheus endpoints are explicitly allowed by the security configuration, avoiding the need for monitoring tools to use a normal user token.

8.2 Circuit Breaker

The project also uses the Circuit Breaker pattern to limit cascading failures during synchronous service calls. The API Gateway applies circuit breakers to the routes that forward traffic to the User, Social, Post, and Notification services. When a downstream service becomes slow or repeatedly fails, the circuit can open and stop sending new calls

for a short period, giving the failing service time to recover and preventing request queues from growing across the system.

Circuit breakers are configured through Resilience4j with count-based sliding windows, failure-rate thresholds, slow-call thresholds, and half-open recovery attempts. Their state is also exposed through Actuator health and circuit-breaker endpoints, so operational tools can observe whether a service dependency is closed, open, or recovering. This pattern complements health checks and metrics: health checks say whether a component is alive, metrics show runtime behavior over time, and circuit breakers actively protect callers from unstable dependencies.

8.3 Application Metrics

Application metrics are collected with Micrometer and exposed in Prometheus format through Spring Actuator. The services expose `/actuator/prometheus`, and each service adds the `application` tag to its metrics so that Prometheus and Grafana can distinguish data coming from the API Gateway, User Service, Social Service, Post Service, and Notification Service.

The project includes both technical and business-oriented metrics. Technical metrics are provided by the Spring Boot and JVM instrumentation and help observe runtime behavior such as HTTP requests, memory usage, and service availability. Business metrics are added where they provide useful insight into the application domain. For example, the User Service exports gauges for registered users, new users created today, and active users in the last five minutes. The Post Service exports gauges derived from the event store, such as total posts, posts created today, comments created today, and reactions created today.

Prometheus is responsible for scraping these metrics from the services, while Grafana is used to visualize them in dashboards. Example Grafana dashboard views are shown in Figure 16 and Figure 17. This combination makes it possible to monitor both infrastructure-level behavior and product-level activity. In practice, this helps detect service failures, observe user growth, and understand whether local community interaction is increasing through posts, comments, and reactions.

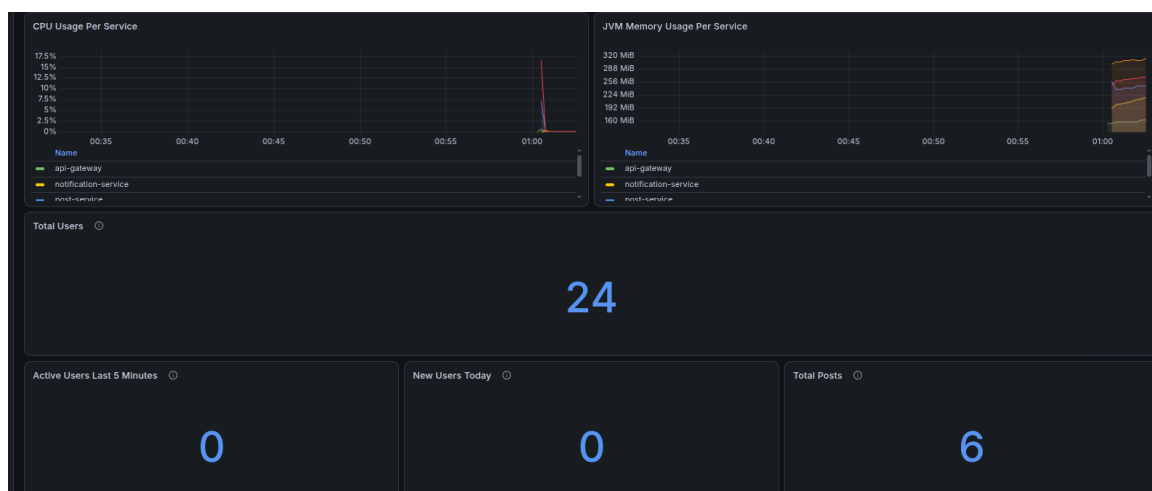


Figure 16: Grafana dashboard example for service and application metrics.

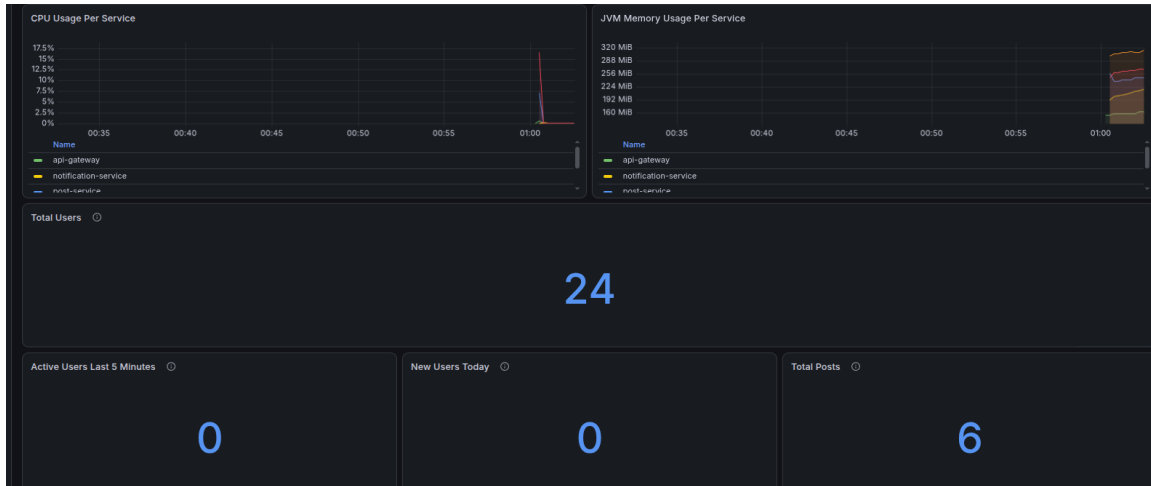


Figure 17: Grafana dashboard example with additional runtime metric panels.

9 Frontend

A lightweight frontend was implemented with React and Vite to make manual end-to-end testing easier. The frontend interacts with the system only through the API Gateway, so it exercises the same public HTTP boundary used by external clients. This made it possible to verify authentication, profile management, neighbour search, neighbour requests, feed browsing, post reactions, comments, notifications, and administrative location management from a single user interface instead of testing each endpoint only through Swagger or direct HTTP calls.

The interface was intentionally kept simple and functional. Its purpose is not to replace the backend tests, but to provide a practical way to validate complete user workflows across multiple services. For example, searching visible neighbours and sending requests can be tested from the view shown in Figure 18; sent and received requests can be inspected from the neighbour management view shown in Figure 19; and post browsing, reactions, and comments can be tested from the feed shown in Figure 20.

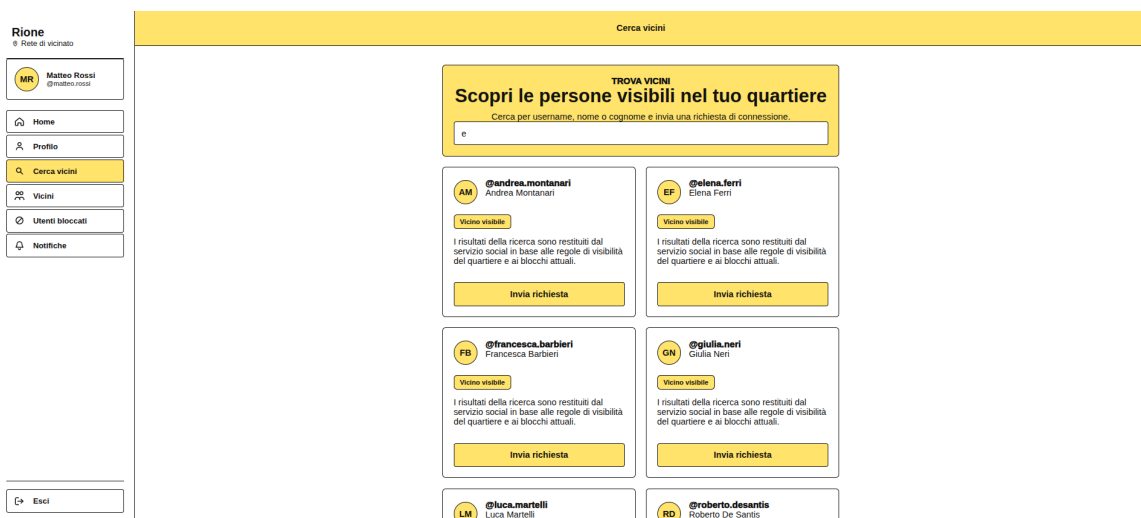


Figure 18: React frontend view for searching visible neighbours and sending neighbour requests.

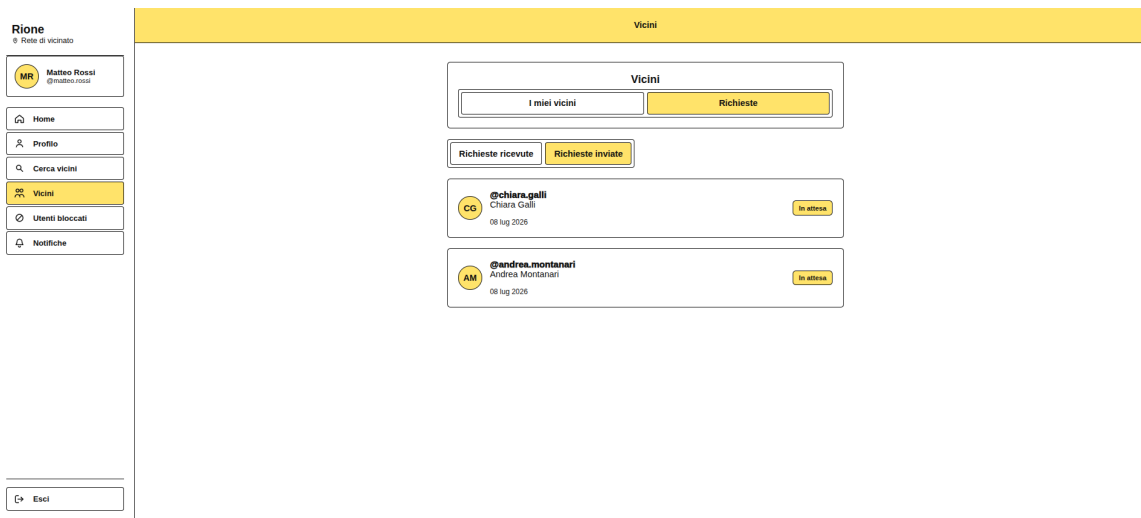


Figure 19: React frontend view for managing neighbours and pending requests.

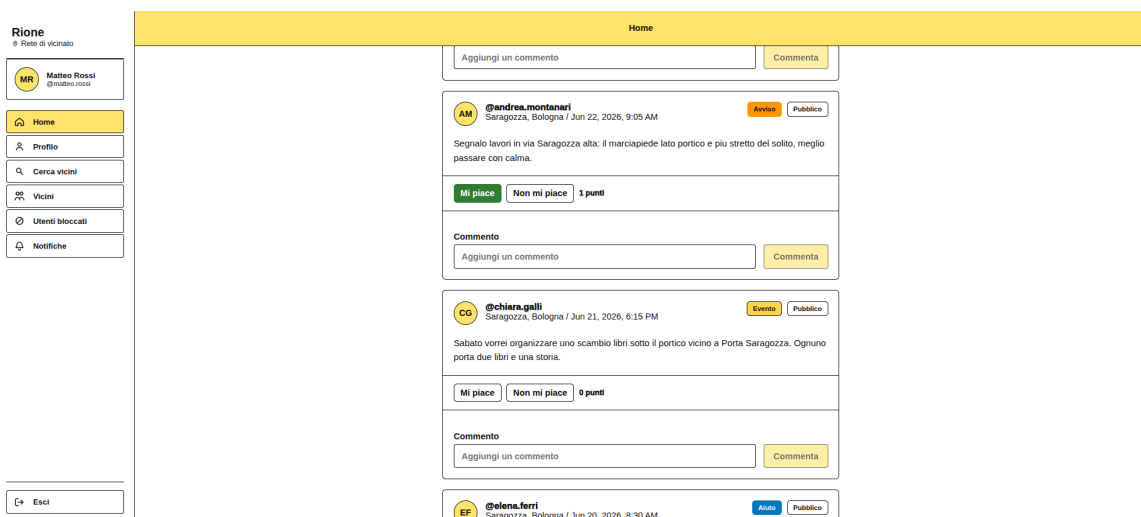


Figure 20: React frontend feed view for posts, reactions, and comments.

10 Implementation

Briefly describe the implementation phase of the project.

10.1 Use of AI During Coding

Briefly describe how AI was used during the coding phase and how its output was reviewed and adapted.

11 Conclusions

The project analysis led to a decomposition that is coherent with both the domain and the expected runtime structure. Starting from the neighbourhood-based social network domain, the system was divided into User, Social, Post, and Notification bounded contexts. This separation made the model easier to reason about because each service owns a focused part of the ubiquitous language and can enforce its own business rules without sharing persistence models or internal domain objects with the others.

From a scalability point of view, this decomposition supports growth along several dimensions. A larger number of users can be handled by scaling the services that receive the most traffic, while the neighbourhood boundary keeps many social interactions local and reduces the need for global coordination. Growth in posts, comments, reactions, and notifications is also handled by separating write-heavy content workflows from notification delivery. Kafka allows notification processing to happen asynchronously, so services that produce events do not need to wait for every notification side effect to complete.

The database-per-service choice also supports scalability because each service can evolve its schema and persistence strategy independently. The Post Service is the most data-intensive component, and event sourcing makes its history explicit while preserving a reliable reconstruction path for current state. At the same time, this choice introduces operational responsibilities: event streams can grow significantly, so future versions should consider snapshots, archival policies, read models, and careful monitoring of replay time.

The architecture also includes operational mechanisms that become important as the system grows. Health checks, Prometheus metrics, Grafana dashboards, and circuit breakers provide visibility and protection in a distributed environment. They help detect service failures, observe usage trends, and prevent synchronous dependencies from propagating failures across services.

The main limitation of the current version is that it is still oriented toward a local Docker Compose deployment rather than a production-grade distributed platform. Future improvements could include service discovery, centralized configuration, stronger deployment automation, horizontal scaling policies, outbox-based event publication, database migration governance, and more complete dashboard definitions. These additions would make the architecture more robust under larger traffic volumes while preserving the domain-driven and hexagonal structure established in the project.